

Algoritmi e Strutture Dati – 13/07/22

Esercizio A1 – Complessità – Punti ≥ 8

Questo esercizio è simile alla funzione di complessità dell'algoritmo deterministico per la selezione che funziona in tempo lineare. Proviamo $\Theta(n)$.

- Visto che la parte induttiva è lineare, la ricorrenza è $\Omega(n)$.
- Limite superiore $O(n)$, dimostrato per sostituzione con induzione. Partendo dal caso base, si ha che:

$$\begin{aligned}T(0) &= 1 \leq c \cdot 0 \Leftrightarrow 1 \leq 0 && \text{Falso!} \\T(1) &= 1 \leq c \cdot 1 \Leftrightarrow c \geq 1 \\T(2) &= 2 + T(\lfloor 4/9 \rfloor) + T(\lfloor 10/18 \rfloor) = 2 + T(0) + T(0) = 4 \leq c \cdot 2 \Leftrightarrow c \geq 4/2 \\T(3) &= 3 + T(\lfloor 6/9 \rfloor) + T(\lfloor 15/18 \rfloor) = 3 + T(0) + T(0) = 5 \leq c \cdot 3 \Leftrightarrow c \geq 5/3 \\T(4) &= 4 + T(\lfloor 8/9 \rfloor) + T(\lfloor 20/18 \rfloor) = 4 + T(0) + T(1) = 6 \leq c \cdot 4 \Leftrightarrow c \geq 4/6 \\T(5) &= 5 + T(\lfloor 10/9 \rfloor) + T(\lfloor 25/18 \rfloor) = 5 + T(1) + T(1)\end{aligned}$$

Abbiamo calcolato tutti questi casi perché la condizione non viene rispettata per il caso $T(0)$, che poi si ripresenta nei casi $T(2)$, $T(3)$ e $T(4)$. A partire da $T(5)$, tuttavia, la ricorrenza utilizza solo casi già dimostrati; possiamo quindi interrompere il calcolo e utilizzare l'induzione.

La soluzione corretta non è stata proposta da nessuno dei partecipanti al compito.

Dimostriamo ora il caso induttivo, assumendo che $T(n') \leq cn'$ per tutti i valori $n' < n$, e volendo dimostrare che $T(n) \leq cn$. Sostituendo abbiamo che

$$\begin{aligned}T(n) &\leq c\lfloor 2n/9 \rfloor + c\lfloor 5n/18 \rfloor + n \\&\leq 2/9n + 5/18cn + n \\&= 9/18cn + n \leq cn \\&= 1/2cn + n \leq cn\end{aligned}$$

L'ultima disequazione è vera per $c \geq 2$. Quindi, per $m = 1, c = \max\{2, 4/2, 5/3, 4/6\} = 2$ abbiamo che $T(n) \leq cn, \forall n \geq m$.

Esercizio A2 – Lago ricorsivo – Punti ≥ 10

È possibile risolvere il problema effettuando una qualunque visita a partire dall'angolo in alto a destra (un punto del mare), e visitando tutti i nodi acqua che possono essere raggiunti nella quattro direzioni verticali e orizzontali. Per comodità utilizziamo il vettore stesso e poniamo a -1 i nodi acqua visitati. Se al termine della visita qualche nodo acqua non è stato visitato, esso fa parte di un lago. La complessità è $O(n^2)$, la dimensione dell'input.

```
boolean containsLake(int[][] M, int n)
```

```
    visitSea(M, n, 1, 1)
    for i = 1 to n do
        for j = 1 to n do
            if M[i][j] == 0 then
                return true
    return false
```

```
boolean visitSea(int[][] M, int i, int j, int n)
```

```
    if 1 ≤ i ≤ n and 1 ≤ j ≤ n and M[i][j] == 0 then
        M[i][j] = -1
        visitSea(M, i - 1, j, n)
        visitSea(M, i, j - 1, n)
        visitSea(M, i + 1, j, n)
        visitSea(M, i, j + 1, n)
```

Per velocità di scrittura, l'algoritmo utilizza una visita DFS ricorsiva; nella realtà, questo porterebbe velocemente all'esaurimento dello stack della chiamate ricorsive, perché nel caso pessimo questo algoritmo visita tutti gli n^2 nodi. Una implementazione più corretta sarebbe basata su una BFS, basata su una coda.

Esercizio A3 – H-index – Punti ≥ 12

Vista la richiesta di risolvere il problema in tempo sub-lineare, utilizziamo la tecnica divide-et-impera. Scriviamo una procedura ricorsiva $\text{hiRec}(A, i, j)$ che restituisce l'h-index, assumendo che sia un valore compreso fra i e j . La procedura viene chiamata con $i = 1$ e $j = n$; essendo valori positivi, non dobbiamo gestire il caso particolare in cui l'h-index è pari a zero (in caso di assenza di citazioni).

Se $i = j$, l'h-index è pari a i , per assunzione che il valore sia compreso fra i e j . Altrimenti, calcoliamo l'elemento centrale m . Se $A[m] \geq m$, allora l'h-index è sicuramente compreso fra m e j ; altrimenti è inferiore a m , e quindi compreso fra i e $m - 1$. Notate l'uso dell'operatore di intero superiore: in questo modo, quando il vettore di riduce a due elementi, si prende il secondo come indice m ; nel caso poi si prenda il sottovettore di sinistra, questo avrà almeno un elemento.

```

int h-index(int[] A, int n)
{
    return hiRec(A, 1, n)
}

int hiRec(int[] A, int i, int j)
{
    if i == j then
        return i
    else
        int m = (i + j) / 2
        if A[m] >= m then
            return hiRec(A, m, j)
        else
            return hiRec(A, i, m - 1)
}

```

La complessità è ovviamente $O(\log n)$.

Esercizio B1 – Prodotto minimo – Punti ≥ 8

Soluzione corretta, basata su greedy Questo è un caso in cui un approccio greedy può funzionare.

L'idea è la seguente:

- Se il vettore contiene valori negativi, è possibile sfruttarli per fare in modo che il prodotto finale sia negativo. Sono dati due casi:
 - se il numero di valori negativi è dispari, si moltiplicano tutti i valori non nulli, ottenendo un prodotto negativo, il più basso possibile;
 - se il numero di valori negativi è pari, si moltiplicano tutti i valori non nulli, tranne il valore negativo maggiore (quello più piccolo in valore assoluto), per evitare che il prodotto sia positivo.

In entrambi i casi, l'obiettivo è ottenere il prodotto più alto in valore assoluto, utilizzando però un numero dispari di valori negativi per ottenere un valore finale negativo.

- Se il vettore contiene solo valori positivi o nulli, si restituisce il minimo valore positivo o nullo (che può essere zero).

La soluzione calcola il minimo valore positivo o nullo $\min_{\geq 0}$ e il massimo valore negativo $\min_{< 0}$, oltre al prodotto di tutti i valori diversi da zero. Se esistono valori negativi ($\max_{< 0} \neq -\infty$), si restituisce il prodotto di tutti i valori non nulli, se negativo (i valori negativi sono dispari); se positivo, si divide per il valore negativo più alto, per rimuovere un valore negativo e rendere il prodotto negativo.